

Business Core as a Service

Modèle Mathématique — Justification et Formalisation

L'architecture de BCaaS repose sur une analogie formelle avec les protocoles réseaux. Pour que cette analogie constitue une méthode de conception rigoureuse et non une simple métaphore, il est nécessaire de lui associer un modèle mathématique qui permette de *prouver* les propriétés du système, de *garantir* son comportement et de *dimensionner* ses ressources.

BCaaS soulève cinq problèmes distincts : la structure du traitement des requêtes à travers les couches, l'isolation des données entre clients, les garanties de performance, la capacité d'absorption du système et le cycle de vie des tenants. Chacun appelle un formalisme différent — aucun modèle unique ne peut les couvrir tous à la fois. C'est pourquoi cinq modèles complémentaires sont présentés ci-après, comme le résume le tableau suivant.

#	Modèle	Problème résolu	Formalisme
1	Composition de fonctions	Comment une requête est-elle transformée en passant par les 5 couches ?	<i>Théorie des fonctions</i>
2	Théorie des ensembles	Comment garantir qu'un tenant ne voit jamais les données d'un autre ?	<i>Partition, intersection vide</i>
3	Logique de prédicats	Comment formaliser et hiérarchiser les garanties de performance ?	<i>Prédicats, ordre total</i>
4	File d'attente M/M/1	Combien de requêtes le système peut-il absorber sans saturer ?	<i>Probabilités, Poisson</i>
5	Suite finie d'états	Comment créer un tenant de façon atomique et sans incohérence ?	<i>Suite, injectivité</i>

Les modèles 1, 2 et 5 sont *structurels* (organisation du système) ; les modèles 3 et 4 sont *opérationnels* (comportement à l'exécution).

0.1 Modèle 1 — Architecture en couches : composition de fonctions

0.1.1 Idée intuitive

Dans BCaaS, une requête HTTP ne va jamais directement au service métier. Elle traverse obligatoirement cinq couches dans un ordre fixe, comme une chaîne de montage industrielle : chaque poste fait une opération précise et transmet le résultat au poste suivant. Le formalisme mathématique naturel pour décrire ce type de traitement séquentiel est la **composition de fonctions**.

0.1.2 Définition formelle

On associe à chaque couche une fonction de transformation :

$$f_1 : r \mapsto r_1 \quad \text{Infrastructure (connexion BD, cache Redis)} \quad (1)$$

$$f_2 : r_1 \mapsto r_2 \quad \text{Transport \& Messaging (REST ou Kafka)} \quad (2)$$

$$f_3 : r_2 \mapsto r_3 \quad \text{Tenant \& Routing (résolution `tenant_id`)} \quad (3)$$

$$f_4 : r_3 \mapsto r_4 \quad \text{Context \& Policy (JWT, RBAC, SLA)} \quad (4)$$

$$f_5 : r_4 \mapsto r_5 \quad \text{Business Capabilities (logique métier)} \quad (5)$$

Le traitement complet d'une requête r est alors la **composée** de ces cinq fonctions :

$$\text{BCaaS}(r) = (f_5 \circ f_4 \circ f_3 \circ f_2 \circ f_1)(r) = f_5\left(f_4\left(f_3\left(f_2\left(f_1(r)\right)\right)\right)\right)$$

0.1.3 Propriété fondamentale : encapsulation

Propriété 0.1.1 (Encapsulation stricte). Pour tout $i \in \{1, \dots, 5\}$, la fonction f_i ne reçoit en entrée que r_{i-1} et ne connaît pas les couches de rang supérieur à i . Formellement :

$$f_i : \mathcal{R}_{i-1} \longrightarrow \mathcal{R}_i \quad \text{et} \quad f_i \text{ est indépendante de } f_j \text{ pour } j > i.$$

C'est la traduction mathématique exacte du principe d'encapsulation du modèle OSI : la couche i ne sait rien de ce qui se passe dans les couches supérieures. Modifier f_5 n'affecte jamais f_1 .

0.1.4 Exemple concret

Un moniteur de l'Auto-École Centrale envoie la requête `POST /api/businesses` pour enregistrer son activité. La requête r entre dans la pile et est enrichie à chaque étape : f_1 ouvre la connexion à la base de données, f_2 choisit le mode REST synchrone, f_3 identifie le tenant via l'en-tête `X-Tenant-ID`, f_4 valide le token JWT et confirme le rôle `ADMIN`, et f_5 persiste l'entité métier et retourne `201 Created`.

Le résultat final s'écrit :

$$\text{BCaaS}(r) = f_5\left(f_4\left(f_3\left(f_2\left(f_1(r)\right)\right)\right)\right) = r_5 = \text{201 Created}.$$

Si f_4 échoue (token JWT expiré), la composition s'arrête à r_4 et retourne `401 Unauthorized`, sans jamais atteindre f_5 .

0.2 Modèle 2 — Isolation multi-tenant : théorie des ensembles

0.2.1 Idée intuitive

Plusieurs clients (tenants) partagent la même instance de PostgreSQL. Pourtant, le client A ne doit jamais voir une donnée du client B . La théorie des ensembles, et plus précisément la notion de **partition**, est le formalisme le plus adapté pour modéliser cette étanchéité.

0.2.2 Définitions

Définition 0.2.1 (Ensemble des tenants). On note $\mathcal{T} = \{t_1, t_2, \dots, t_n\}$ l'ensemble fini de tous les tenants enregistrés sur la plateforme.

Définition 0.2.2 (Ensemble des entités). On note $\mathcal{E}$ l'ensemble de toutes les lignes stockées en base de données (toutes tables métier confondues).

Définition 0.2.3 (Fonction d'appartenance). On définit la fonction $\pi : \mathcal{E} \rightarrow \mathcal{T}$ qui associe à chaque entité $e \in \mathcal{E}$ le tenant auquel elle appartient :

$$\pi(e) = t_k \iff \text{la colonne } \texttt{tenant_id} \text{ de } e \text{ vaut } t_k.$$

L'ensemble des données du tenant t_k est alors :

$$\mathcal{D}(t_k) = \{ e \in \mathcal{E} \mid \pi(e) = t_k \}.$$

0.2.3 Propriété d'isolation

Propriété 0.2.1 (Isolation totale entre tenants).

$$\forall t_i, t_j \in \mathcal{T}, \quad t_i \neq t_j \implies \mathcal{D}(t_i) \cap \mathcal{D}(t_j) = \emptyset.$$

Cette propriété est garantie dans le code par le `TenantFilter` de Spring Security, qui injecte le `tenant_id` dans chaque requête SQL via un `ThreadLocal`.

0.2.4 Structure de partition

Les ensembles $\mathcal{D}(t_1), \mathcal{D}(t_2), \dots, \mathcal{D}(t_n)$ forment une **partition** de $\mathcal{E}$:

$$\mathcal{E} = \biguplus_{k=1}^n \mathcal{D}(t_k) = \mathcal{D}(t_1) \uplus \mathcal{D}(t_2) \uplus \dots \uplus \mathcal{D}(t_n)$$

Le symbole $\uplus$ désigne l'union *disjointe* : les ensembles sont deux à deux disjoints et leur union reconstitue la totalité des données. Aucune entité n'est « sans tenant » ni « dans deux tenants à la fois ».

0.2.5 Exemple concret

Deux entreprises sont inscrites sur BCaaS : l'Auto-École Centrale (t_1) et la Clinique Mélen (t_2). Chacune a créé ses propres dossiers : l'auto-école possède les entités e_1, e_2, e_3 (véhicules, moniteurs, élèves) et la clinique les entités e_4, e_5 (patients, consultations).

La partition s'écrit :

$$\mathcal{D}(t_1) = \{e_1, e_2, e_3\} \quad \text{et} \quad \mathcal{D}(t_2) = \{e_4, e_5\}, \quad \mathcal{D}(t_1) \cap \mathcal{D}(t_2) = \emptyset.$$

Un médecin de la clinique, authentifié sous t_2 , ne pourra jamais lire e_1, e_2 ou e_3 : le `TenantFilter` ajoute automatiquement `WHERE tenant_id = 't2'` à chaque requête SQL, rendant les données de t_1 physiquement invisibles.

0.3 Modèle 3 — Qualité de service : logique de prédicats

0.3.1 Idée intuitive

BCaaS propose quatre niveaux de service différenciés, inspirés des classes DiffServ des réseaux IP. Un tenant PLATINUM paie davantage mais bénéficie d’une latence plus faible et d’un débit plus élevé. Il faut un formalisme capable d’exprimer des **conditions** (le SLA est-il respecté?) et une **hiérarchie stricte** entre niveaux.

0.3.2 Les quatre niveaux de service

$$\mathcal{Q} = \{ \text{BRONZE, SILVER, GOLD, PLATINUM} \}$$

Niveau	$\tau_{\max}(q)$	$\lambda_{\min}(q)$	Dispo. ϱ_q	Isolation
BRONZE	5 s	10 r/s	Standard	Colonne <code>tenant_id</code>
SILVER	2 s	50 r/s	Prioritaire	Col. + Redis
GOLD	500 ms	200 r/s	24/7 dédié	Schéma dédié
PLATINUM	100 ms	Illim.	Contractuelle	Base dédiée

0.3.3 Le prédicat SLA_OK

Définition 0.3.1 (Respect du SLA). Pour une transaction x et un niveau de service $q \in \mathcal{Q}$, le SLA est **respecté** si et seulement si les deux conditions suivantes sont simultanément vraies :

$$\text{SLA_OK}(x, q) \iff \underbrace{\tau(x) \leq \tau_{\max}(q)}_{\text{latence dans la limite}} \wedge \underbrace{\Lambda \geq \lambda_{\min}(q)}_{\text{débit suffisant}}$$

où $\tau(x)$ est la latence observée de la transaction x et Λ le débit d’entrée courant (req/s).

0.3.4 Relation d’ordre total entre niveaux

Les quatre niveaux sont munis d’une **relation d’ordre total** strict :

$$\text{BRONZE} \prec \text{SILVER} \prec \text{GOLD} \prec \text{PLATINUM}$$

Cet ordre est **compatible** avec les contraintes de performance :

Propriété 0.3.1 (Monotonie des garanties). Pour tout $q_1, q_2 \in \mathcal{Q}$:

$$q_1 \prec q_2 \implies \tau_{\max}(q_1) > \tau_{\max}(q_2) \wedge \lambda_{\min}(q_1) < \lambda_{\min}(q_2).$$

En clair : un niveau supérieur tolère une latence *strictement plus courte* et garantit un débit *strictement plus élevé*.

0.3.5 Exemple concret

La Clinique Mélen est abonnée au niveau **SILVER** ($\tau_{\max} = 2\text{ s}$, $\lambda_{\min} = 50\text{ req/s}$). Un médecin crée un dossier patient : la requête prend $\tau(x) = 1,8\text{ s}$ et le débit courant est $\Lambda = 30\text{ req/s}$.

$$\text{SLA_OK}(x, \text{SILVER}) \iff \underbrace{1,8 \leq 2}_{\text{vrai}} \wedge \underbrace{30 \geq 50}_{\text{faux}} = \text{faux}.$$

La latence est correcte, mais le débit est insuffisant : le SLA est **violé**. BCaaS émet une alerte et le contrat prévoit un remboursement partiel pour la clinique.

0.4 Modèle 4 — Capacité du système : file d'attente M/M/1

0.4.1 Idée intuitive

BCaaS est un serveur web : des requêtes arrivent de façon aléatoire et Spring Boot les traite une par une par thread. Pour garantir les SLA du modèle 3, il faut répondre à la question : *combien de requêtes le système peut-il absorber sans que la latence dépasse $\tau_{\max}(q)$?* La théorie des files d'attente apporte une réponse analytique exacte.

0.4.2 Modélisation M/M/1

On modélise le système comme une file **M/M/1** : les arrivées suivent un processus de Poisson de taux Λ (req/s) et les durées de traitement suivent une loi exponentielle de paramètre μ (req/s).

Définition 0.4.1 (Taux d'utilisation).

$$\varrho = \frac{\Lambda}{\mu} \in [0, 1)$$

ϱ représente la fraction du temps où le serveur est occupé.

0.4.3 Les trois résultats fondamentaux

Condition de stabilité (le système ne sature pas) :

$$\varrho = \frac{\Lambda}{\mu} < 1 \iff \Lambda < \mu$$

Si $\varrho \geq 1$, la file grossit indéfiniment et le système s'effondre.

Latence moyenne (temps d'attente + temps de traitement) :

$$\bar{\tau} = \frac{1}{\mu - \Lambda}$$

Nombre moyen de requêtes en cours (loi de Little) :

$$N = \Lambda \cdot \bar{\tau} = \frac{\varrho}{1 - \varrho} = \frac{\Lambda}{\mu - \Lambda}$$

0.4.4 Application : dimensionnement pour le SLA GOLD

Le niveau GOLD exige $\tau_{\max}(\text{GOLD}) = 500 \text{ ms} = 0,5 \text{ s}$. Pour un taux d'arrivée $\Lambda = 40 \text{ req/s}$, on calcule la capacité minimale μ nécessaire :

Étape	Calcul	Résultat
1	On part de $\bar{\tau} \leq 0,5 \text{ s}$	
2	On remplace : $\frac{1}{\mu - \Lambda} \leq 0,5$	
3	On inverse : $\mu - \Lambda \geq \frac{1}{0,5} = 2$	
4	On conclut : $\mu \geq \Lambda + 2 = 40 + 2$	$\mu \geq 42 \text{ req/s}$

Conclusion pratique : pour respecter le SLA GOLD ($\tau_{\max} = 500 \text{ ms}$) avec un taux d'arrivée de 40 req/s , le pool de threads Spring Boot doit être dimensionné pour traiter au minimum $\mu \geq 42 \text{ req/s}$. Généralisation :

$$\mu_{\min}(q) = \Lambda + \frac{1}{\tau_{\max}(q)}.$$

0.5 Modèle 5 — Provisionnement d'un tenant : suite d'états

0.5.1 Idée intuitive

Quand une entreprise s'inscrit sur BCaaS, le système doit exécuter plusieurs opérations dans un ordre précis et sans interruption partielle. Si l'étape 3 échoue, on ne doit pas se retrouver avec un tenant à moitié créé. On modélise ce processus comme une **suite finie d'états** avec des préconditions formelles.

0.5.2 Les sept états du provisionnement

Soit $S = (S_0, S_1, \dots, S_6)$ la suite des états :

État	Nom	Description et précondition formelle
S_0	Requête reçue	Réception de POST /api/tenants.
S_1	Validation domaine	Précondition : $\text{dom}_{\text{req}} \notin \{ \text{dom}(t) \mid t \in \mathcal{T} \}$. Sinon : erreur 409.
S_2	Génération id	On génère $\text{id} \in \text{UUID}_4$. Probabilité de collision $\approx 10^{-36}$.
S_3	Insertion en base	Transaction SQL atomique dans la table <code>tenant</code> .
S_4	Contexte isolation	Initialisation de $\mathcal{D}(t_{\text{new}}) = \emptyset$. Le <code>TenantFilter</code> est configuré.
S_5	Génération JWT	Token signé : $(\text{tenant_id}, \text{roles}, \text{exp})$. $\text{exp} = t_{\text{now}} + 24 \text{ h}$ (analogie TTL).
S_6	Tenant actif	$t_{n+1} \in \mathcal{T}$ et $\mathcal{D}(t_{n+1}) = \emptyset$.

0.5.3 Propriété d'injectivité du domaine

La précondition de S_1 garantit une propriété algébrique importante :

Propriété 0.5.1 (Injectivité de la fonction domaine). La fonction qui associe à chaque tenant son nom de domaine est **injective** :

$\text{domaine} : \mathcal{T} \longrightarrow \text{Dom}$ est injective

c'est-à-dire : $\text{domaine}(t_i) = \text{domaine}(t_j) \implies t_i = t_j$.

En d'autres termes : deux tenants distincts ne peuvent jamais partager le même nom de domaine.

Cette propriété est analogue à l'unicité des adresses IP sur un réseau : chaque nœud (tenant) possède une adresse (domaine) unique qui permet de le router sans ambiguïté.

0.5.4 Exemple concret

Un gestionnaire inscrit le **Cabinet Dupont** sur BCaaS via `POST /api/tenants {"domain":"cabinet`

En S_1 , le système vérifie que **cabinet-dupont** n'est pas déjà pris dans $\mathcal{T}$: la précondition est satisfaite, on continue. En S_2 , un identifiant unique est généré : `id = "b9c2-..."`. En S_3 , la ligne est insérée en base de façon atomique. En S_4 , l'espace de données $\mathcal{D}(t_{\text{new}}) = \emptyset$ est initialisé. En S_5 , un token JWT est émis. En S_6 , le Cabinet Dupont est officiellement dans $\mathcal{T}$:

$$t_{n+1} \in \mathcal{T}, \quad \mathcal{D}(t_{n+1}) = \emptyset.$$

Si une seconde requête tentait le même domaine, S_1 retournerait immédiatement **409 Conflict**, car l'injectivité de la fonction *domaine* serait violée.

Synthèse et cohérence du modèle

Les cinq modèles présentés dans ce chapitre ne sont pas des formalismes indépendants juxtaposés : ils forment un tout cohérent dont chaque élément s'appuie sur les précédents et prépare les suivants.

Récapitulatif des propriétés formelles

#	Modèle	Propriété formelle centrale
1	Composition	$BCaaS(r) = (f_5 \circ f_4 \circ f_3 \circ f_2 \circ f_1)(r)$ — f_i indépendante de f_j pour $j > i$
2	Ensembles	$\mathcal{D}(t_i) \cap \mathcal{D}(t_j) = \emptyset$ pour $t_i \neq t_j$ — $\mathcal{E} = \biguplus_{k=1}^n \mathcal{D}(t_k)$
3	Prédicats	$SLA_OK(x, q) \iff \tau(x) \leq \tau_{\max}(q) \wedge \Lambda \geq \lambda_{\min}(q)$ — $BRONZE \prec \dots \prec PLATINUM$
4	File M/M/1	$\bar{\tau} = \frac{1}{\mu - \Lambda}$ — $\mu_{\min}(q) = \Lambda + \frac{1}{\tau_{\max}(q)}$
5	Suite d'états	domaine : $\mathcal{T} \rightarrow Dom$ est injective — création atomique en 7 états

Articulation entre les modèles

Les cinq modèles s'articulent selon deux axes : un axe **structurel** (comment le système est organisé) et un axe **opérationnel** (comment le système se comporte à l'exécution).

Axe structurel — Modèles 1, 2 et 5. Le modèle 1 décrit la *forme* du traitement : toute requête traverse cinq couches en séquence, chacune ignorant les niveaux supérieurs. Le modèle 2 décrit la *structure des données* : l'espace de stockage est partitionné entre tenants de façon étanche. Le modèle 5 décrit la *naissance* d'un tenant : l'entrée d'un nouvel élément dans la partition se fait via une suite d'états atomiques qui garantit l'unicité du domaine par injectivité.

Axe opérationnel — Modèles 3 et 4. Le modèle 3 fixe les *contrats de performance* : pour chaque niveau de service, des bornes sur la latence et le débit sont formellement

définies et ordonnées. Le modèle 4 traduit ces contrats en *contraintes de dimensionnement* physique : la formule $\mu_{\min}(q) = \Lambda + \frac{1}{\tau_{\max}(q)}$ donne directement la capacité de traitement minimale à provisionner dans Spring Boot pour que les prédicats du modèle 3 soient satisfaits.

Cohérence globale du modèle. Le modèle 1 pose le cadre d'exécution. Le modèle 2 garantit l'isolation des données dans ce cadre. Le modèle 3 exprime ce que le système *promet* à chaque tenant. Le modèle 4 calcule ce que le système *doit* être capable de faire pour tenir ces promesses. Le modèle 5 assure enfin que chaque tenant entre dans ce système de façon *cohérente et sans ambiguïté*. Ensemble, ces cinq formalismes constituent une spécification mathématique complète de BCaaS.